KG Finance

Project Overview, Technology Stack, Calculations & Workflows

App name: KG Finance (formerly Grik Finance)

Users: Kushvanth & Grishma

Stack: Next.js 16, React 19, TypeScript, Zustand, Supabase

Document date: July 2026

1. What Is This Project?

KG Finance is a personal finance web app built for a couple. It tracks income, spending, bank/credit accounts, debts, and a shared **Between Us** balance when one person pays for the other.

Feature	Purpose
Dashboard	Monthly KPIs, bar/donut/line charts, tap-to-see payment lists
Spend	Record bills, debt payments, and custom expenses
Between Us	Shared ledger — who paid for whom and who owes back
Income	Paychecks and income by source
Expenses	Recurring monthly bill templates
Accounts	Cash, debit, and credit card balances
Debts	Loans and credit card debt tracking
History	Full transaction log + deleted audit trail
Cloud Sync	Sync data between devices via Supabase

> **Important:** This app does not use a traditional server-side database for calculations. Almost all logic runs **in the browser**. Supabase stores and syncs a JSON snapshot of the full finance state.

2. Technology Stack

Frontend

Technology	Role
Next.js 16	App framework (App Router, pages, API route)
React 19	UI components
TypeScript	Type safety across the codebase
Tailwind CSS 4	Styling (glass UI, dark/light mode)
Zustand	Global state management (finance-store)
Recharts	Dashboard charts
date-fns	Date/month handling
lucide-react	Icons
uuid	Unique IDs for records

Data & Sync

Technology	Role
localStorage	Primary persistence on each device (Zustand persist)
Supabase	Cloud JSON blob + realtime sync between devices
/api/sync-config	Returns Supabase URL and anon key to the client

3. Core Data Model

Everything is stored in one `FinanceState` object:

```
accounts[]           → cash / debit / credit balances
debts[]              → loan & credit card amounts
incomeSources[]      → e.g. "Salary", "Freelance"
incomeEntries[]      → actual paycheck deposits
monthlyExpenses[]    → bill templates (rent, subscriptions, etc.)
transactions[]       → every money movement (history)
interCoupleHistory[] → Between Us ledger entries
interCoupleBalance   → current who-owes-who number
deletedHistory[]     → audit log when transactions are deleted
```

Two people only: `kushvanth` | `grishma`

4. Calculations Reference

4.1 Income

```
Monthly Income = SUM(income entries where person = X and date in that month)
Yearly Income  = SUM(income entries in calendar year)
By source      = GROUP BY income source name
```

4.2 Expense Share (Split Bills)

```
if expenseShares[person] exists     → use that share
else if expenseOwner/beneficiary = person → full amount
else                                → 0
```

4.3 Monthly Spending (Dashboard)

```
Debt & Between Us =
  SUM(debt_payment + credit_payment where person paid)
  + SUM(inter_couple where person paid)
  + Between Us history entries (avoid double-count with linked transactions)

Other spending = SUM(expense share for person this month)

Total spending = Debt & Between Us + Other spending
```

4.4 Cash Flow & Savings

```
Cash flow = Monthly Income − Total Spending
Saved     = max(0, Cash flow)
Over budget = abs(Cash flow) when negative
```

4.5 Between Us Balance

```
If Kushvanth paid for Grishma → balance += amount
If Grishma paid for Kushvanth → balance -= amount

balance > 0  → Grishma owes Kushvanth
balance < 0  → Kushvanth owes Grishma
balance = 0  → even
```

4.6 Accounts & Net Worth

```
Account assets = SUM(debit + cash balances)   // credit excluded as asset
Total debt     = SUM(debt.amount)
Net worth      = assets − liabilities

Credit utilization = (balance / creditLimit) × 100
Available credit   = creditLimit − balance
```

4.7 Bill Progress (Spend Page)

```
paidThisMonth = SUM(expenses linked to that bill this month)
remaining     = max(0, plannedAmount − paidThisMonth)
```

4.8 Month-over-Month Trend

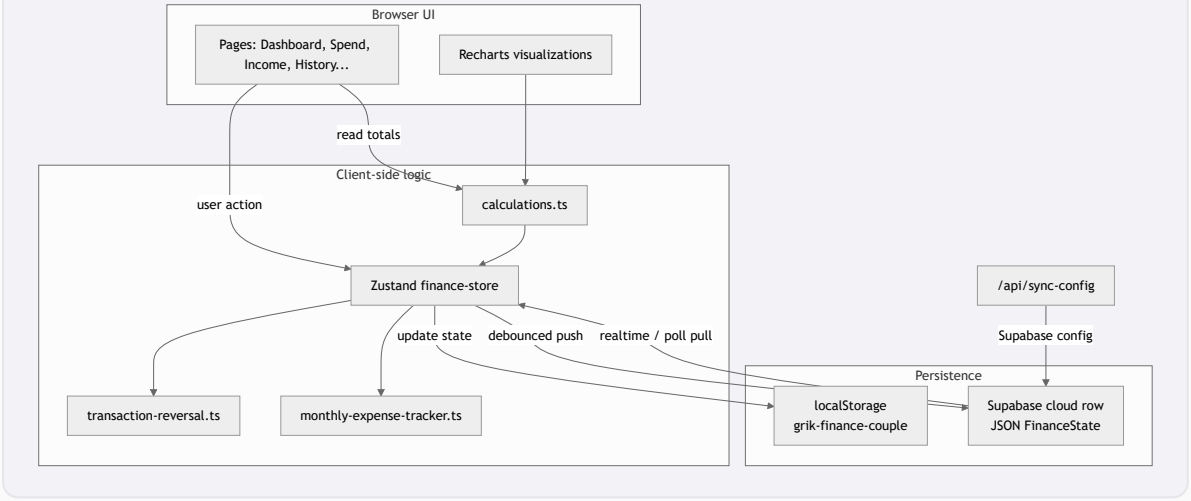
```
spendChange% = ((thisMonthSpend − lastMonthSpend) / lastMonthSpend) × 100
```

4.9 Payment from Account (When Spending)

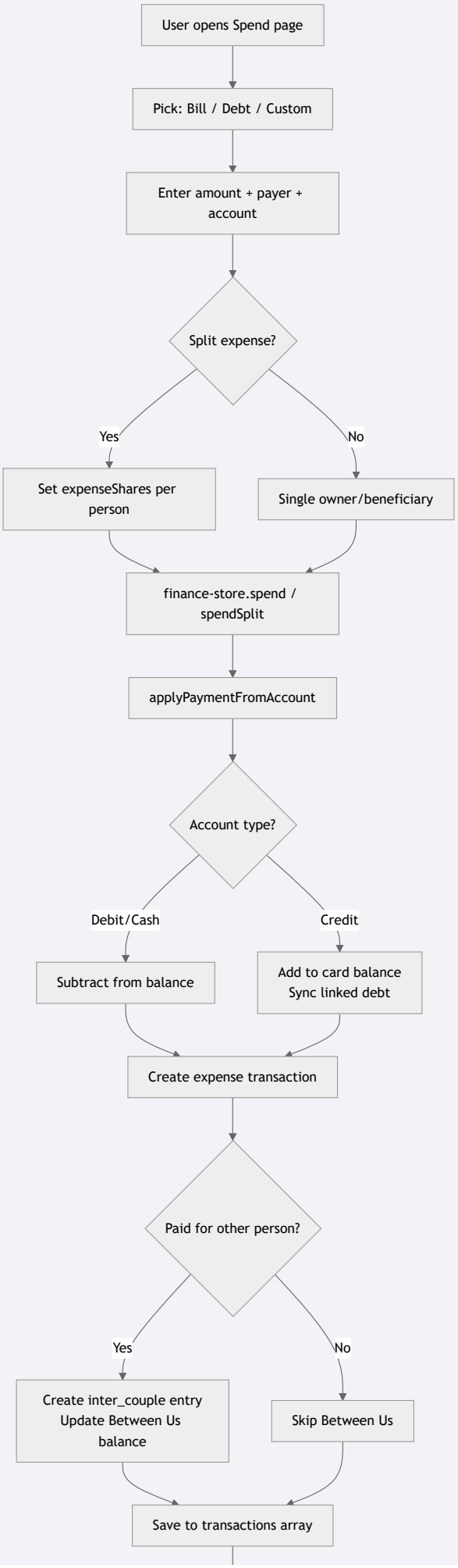
```
Debit/Cash  → subtract from balance
Credit      → add to card balance (debt increases)
Cash from debit → subtract from both accounts
```

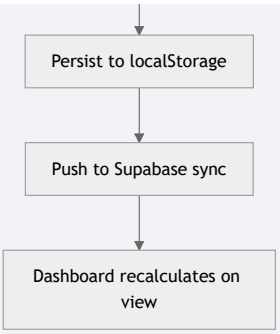
5. Workflow Diagrams

5.1 Overall App Architecture

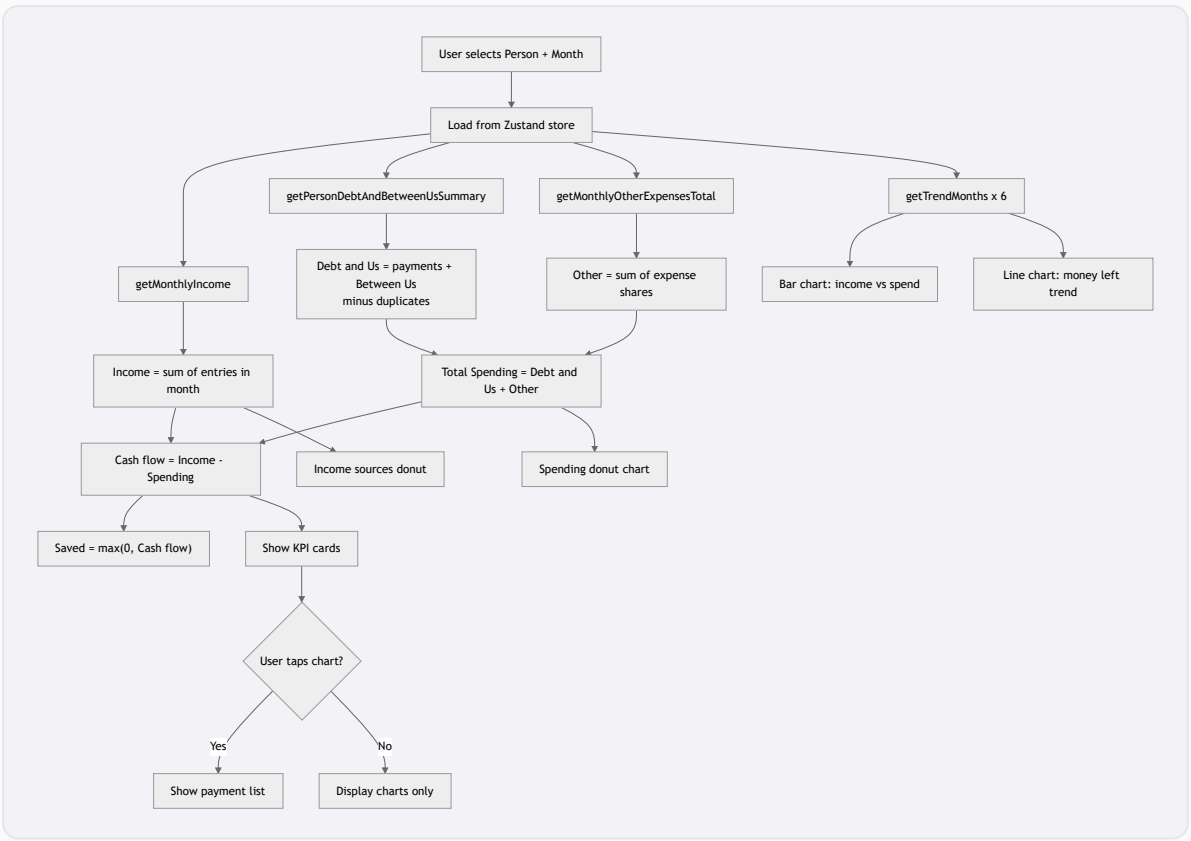


5.2 Recording a Spend Transaction


User opens Spend page
Pick: Bill / Debt / Custom
Enter amount + payer + account
Split expense?
Yes
No
Set expenseShares per person
Single owner/beneficiary
finance-store.spend / spendSplit
applyPaymentFromAccount
Account type?
Debit/Cash
Credit
Subtract from balance
Add to card balance
Sync linked debt
Create expense transaction
Paid for other person?
Yes
No
Create inter_couple entry
Update Between Us balance
Skip Between Us
Save to transactions array


Persist to localStorage
Push to Supabase sync
Dashboard recalculates on view

5.3 Dashboard Calculation Pipeline



5.4 Cloud Sync Flow

6. Key Source Files

File	Purpose
src/store/finance-store.ts	All write operations: spend, income, pay debt, delete
src/lib/calculations.ts	Read-only math for dashboard and reports
src/lib/transaction-reversal.ts	Undo deletes, recalculate Between Us
src/lib/monthly-expense-tracker.ts	Bill paid / remaining this month
src/lib/inter-couple.ts	Between Us labels and transaction linking
src/lib/supabase/sync.ts	Push/pull cloud state
src/components/dashboard/	Charts and click-to-list UI

7. Summary

KG Finance is a Next.js couple budgeting app that stores all financial data locally (and optionally in Supabase), runs every calculation in the browser when you view or change data, and keeps a shared Between Us ledger so split and cross-person payments stay fair.
